

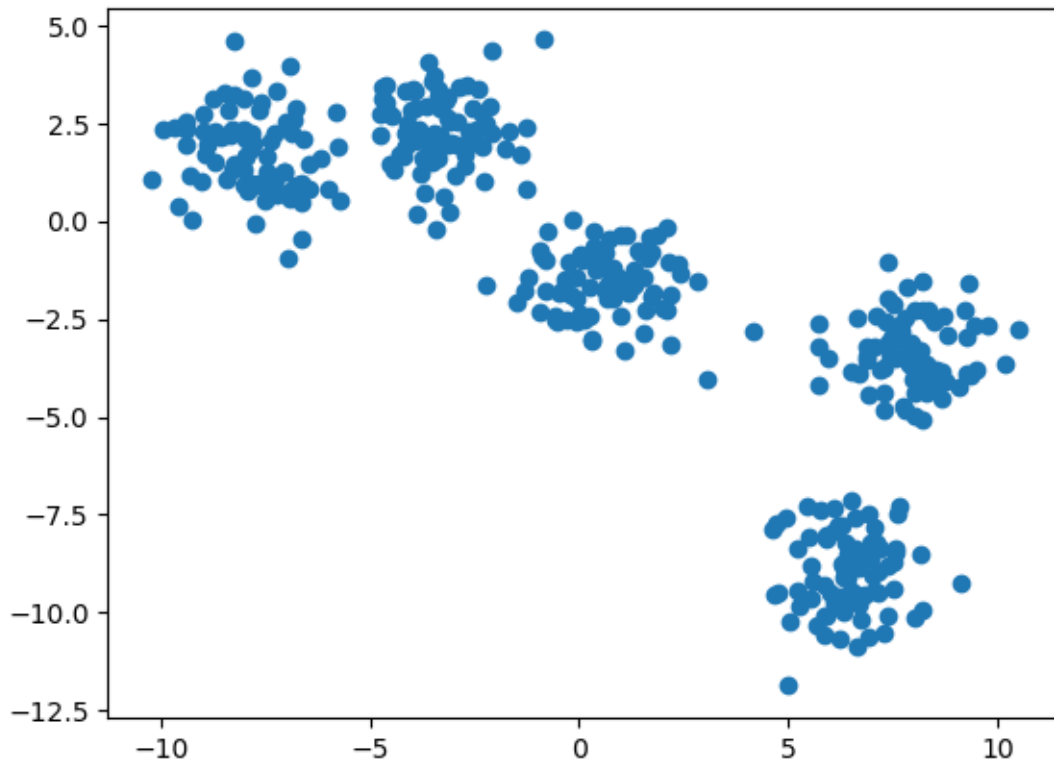
Medir el desempeño

Guillermo Ruiz

Vamos a usar k-means para crear los clusters y luego vamos usar diferentes métricas para medir la calidad de los clusters.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
```

```
X, y = make_blobs(400, 2, centers=5, random_state=6)
plt.scatter(X[:, 0], X[:, 1])
plt.show()
```



Suma de cuadrados

La suma de los cuadrados de las distancias se guarda en el atributo `inertia_`. Vamos a usar varios valores de k y mostrar la suma de cuadrados para comparar.

```
K = [2, 3, 4, 5, 6, 7, 20, 200]
for k in K:
    km = KMeans(n_clusters=k, n_init='auto')
    km.fit(X)
    print(f"k: {k} SSE:{km.inertia_}")
```

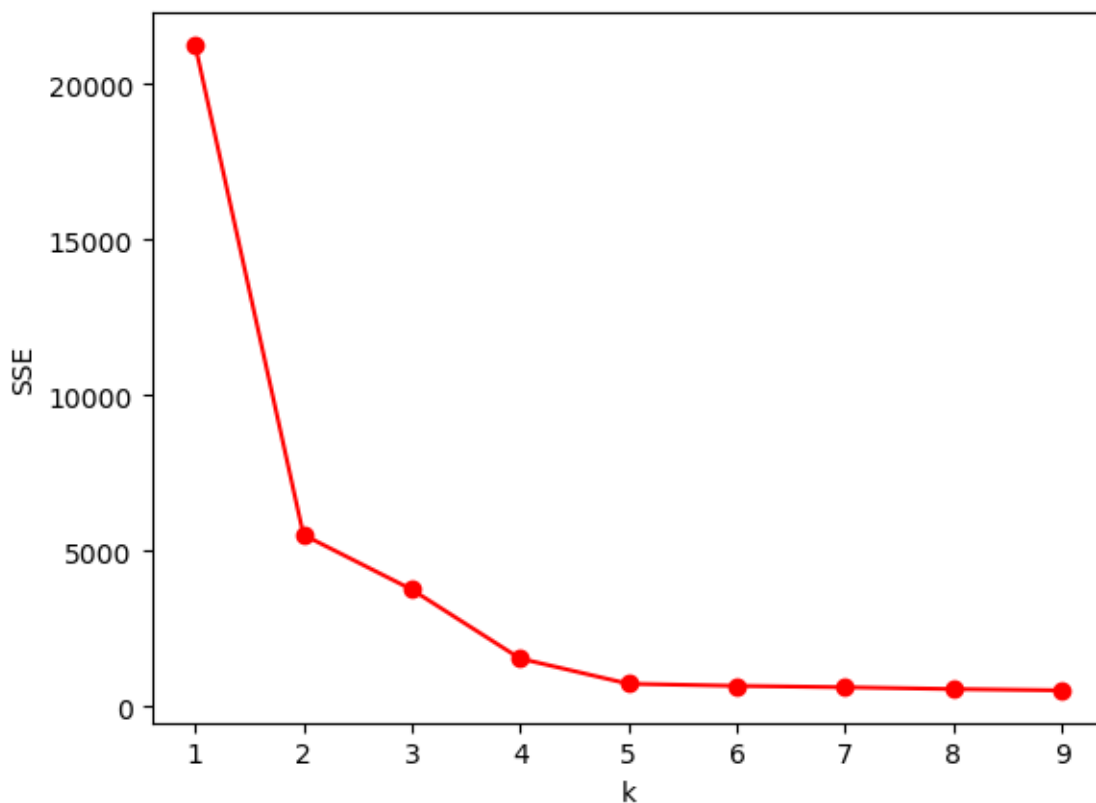
```
k: 2 SSE:5513.41689566692
k: 3 SSE:2880.572119348002
k: 4 SSE:1538.6060106168538
k: 5 SSE:727.7950589404811
k: 6 SSE:660.1055286250289
k: 7 SSE:592.7161408460752
k: 20 SSE:245.8191667466011
k: 200 SSE:6.251176758648581
```

Note cómo el valor disminuye cuando la k crece. Graficar el error puede darnos una idea del número correcto de clusters que podemos usar.

```
sse = []
k_range = list(range(1, 10))

for k in k_range:
    km = KMeans(n_clusters=k, n_init='auto')
    km.fit(X)
    sse.append(km.inertia_)

plt.plot(k_range, sse, '-or')
plt.xlabel("k")
plt.ylabel('SSE');
```



Este gráfico nos muestra la **técnica del codo**. La técnica del codo nos puede ayudar a elegir el mejor número de clusters. Se recomienda elegir el número que corresponde a la parte curva de la gráfica, donde estaría el “codo”.

Actividad: variar el conjunto de datos y comparar los resultados.

Silueta

Ahora veremos la métrica de la silueta. Para esto, usaremos la función `silhouette_score`.

```
from sklearn.metrics import silhouette_score
```

```
K = [2,3,4,5,6,7,8,9]
for k in K:
    km = KMeans(k, n_init='auto')
    y_km = km.fit_predict(X)
    score = silhouette_score(X, y_km, metric='euclidean')
    print(f"k: {k} Silueta:{score}")
```

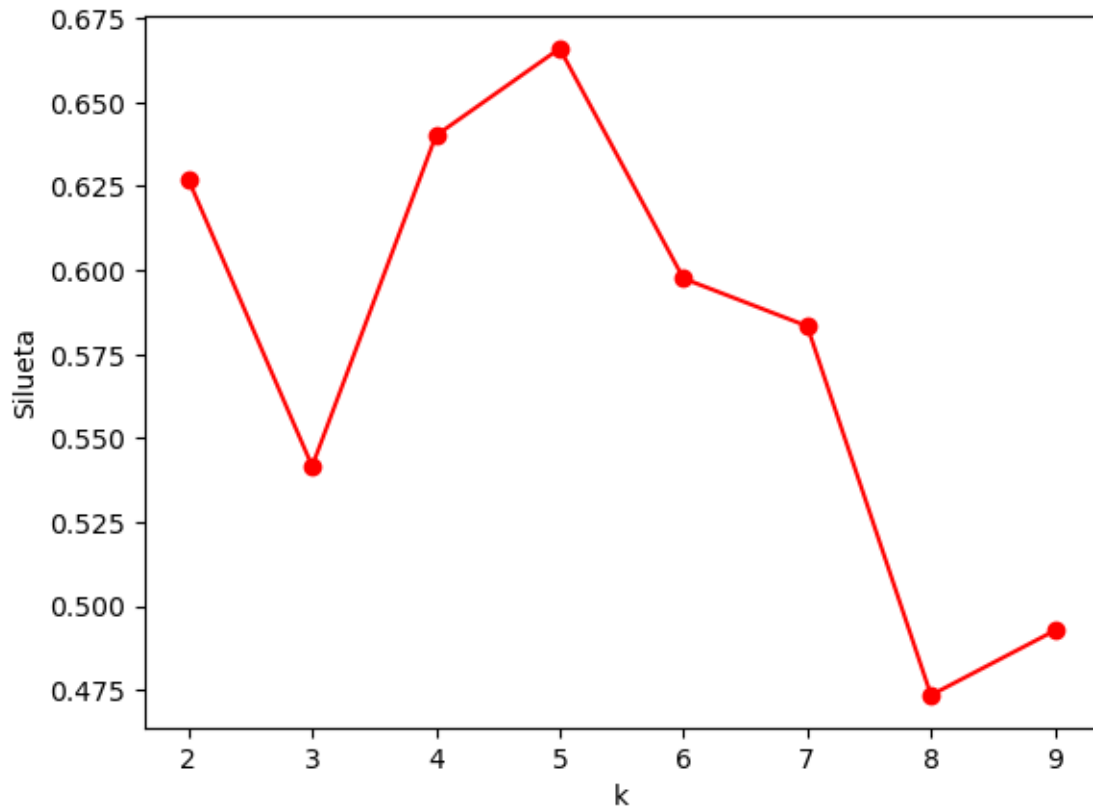
```
k: 2 Silueta:0.6259053739770747
k: 3 Silueta:0.579192119232036
k: 4 Silueta:0.6400389444077607
k: 5 Silueta:0.6659658730536087
k: 6 Silueta:0.5897338358321599
k: 7 Silueta:0.5086870606721661
k: 8 Silueta:0.45865965437815104
k: 9 Silueta:0.46654949721619693
```

De la misma manera, podemos graficar los valores para saber el mejor valor de `k` para nuestros datos.

```
sil = []
k_range = list(range(2, 10))

for k in k_range:
    km = KMeans(n_clusters=k, n_init='auto')
    y_km = km.fit_predict(X)
    score = silhouette_score(X, y_km, metric='euclidean')
    sil.append(score)

plt.plot(k_range, sil, '-or')
plt.xlabel("k")
plt.ylabel('Silueta')
plt.show()
```



Mientras más alto, la separación el clusters se considera mejor. Podemos ver que en este caso el valor de 5 está correctamente estimado.

Actividad: variar el conjunto de datos y comparar los resultados.